PYTHON PROGRAMMING

ASSIGNMENT 1

Design, Implement and Validate a Payment Calculator for a Village Milk Co-operative

TASK 1 – Understand the Problem and Identify the Data (4 Marks)

The calculator reads farmer ID, litres, fat percentage, SNF percentage and shift. It computes rate per litre, individual payment and ten-day total. The reference rate is ₹30.00/L at 3.5% fat and 8.5% SNF. Every 0.1 percentage-point fat step changes the rate by ₹0.40, and every 0.1 SNF step changes it by ₹0.30.

Value	Type	Justification and rule
Farmer ID	str	Can contain letters or leading zeroes; cannot be blank.
Litres	Decimal	May be fractional; must be greater than zero.
Fat %	Decimal	Fractional reading; allowed range 2.0–7.0 inclusive.
SNF %	Decimal	Fractional reading; assumed 6.0–10.0 here; confirm official range.
Shift	str	Accept AM or PM only.
Step counts	int	Whole-number count of 0.1 percentage-point steps.
Rate/payment/total	Decimal	Currency values rounded to two decimal places.

Named constants

```
BASE_RATE = Decimal("30.00")
REFERENCE_FAT = Decimal("3.5")
REFERENCE_SNF = Decimal("8.5")
PERCENT_STEP = Decimal("0.1")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
MIN_FAT, MAX_FAT = Decimal("2.0"), Decimal("7.0")
```

- Named constants make rate revisions controlled and consistent.
- Assume nearest whole 0.1 percentage-point steps.
- Round each farmer payment to paise before adding the ten-day total.
- Reject non-numeric inputs, non-positive litres, out-of-range readings and invalid shifts.

Alternative concept and complexity

Using litres1, fat1, snf1, litres2, fat2, snf2, etc. creates hundreds of separate names. For 300 farmers there are already 900 measurement variables. A list of records lets one loop handle any number n.

Design	Time	Space	Assessment
Separate variables	Repetitive; code grows with n	O(n) values	Difficult to maintain
List of records	O(n)	O(n)	Flexible and auditable
Running total only	O(n)	O(1) extra	Best if individual records need not be retained

TASK 2 – Explore Values and Types in the Interpreter (4 Marks)

Interactive session

```
>>> type(10)
<class 'int'>
>>> type(10.0)
<class 'float'>
>>> type("10")
<class 'str'>
>>> type(True)
<class 'bool'>
>>> type([10, 8.5])
<class 'list'>
>>> 7 / 2
3.5
>>> 7 // 2
```

```
3
>>> 7 % 2
1
>>> -7 // 2
-4
>>> 0.1 + 0.2 == 0.3
False
```

int is a whole number, float is binary floating point, str is text, bool is True/False, and list stores a sequence. `/` is true division; `//` floors; `%` returns the remainder. -7 // 2 is -4 because floor division rounds down toward negative infinity. Binary floating point cannot exactly represent many decimal fractions, so 0.1 + 0.2 is not exactly 0.3.

Complete the Code – Decimal total

```
from decimal import Decimal

payments = ["33.40"] * 1000 + ["0.10"] * 3

def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total

def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total

print("float  :", total_float(payments))
print("Decimal:", total_decimal(payments))
print(0.1 + 0.2 == 0.3,
      Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))


Expected output:
float  : 33400.30000000061
Decimal: 33400.30
False True
```

Use Decimal(amount) when amount is a string. Decimal(float(amount)) first converts a binary float approximation and can show many extra digits. Alternatives are integer paise (₹33.40 = 3340 paise) or Decimal with a documented rounding rule. Repeated round() on floats does not guarantee an exact bank total.

Method	Time per addition	Exact bank total?	Memory
Decimal from string	O(1) for fixed-size amounts	Yes, with defined rounding	Measure using sys.getsizeof()
Integer paise	O(1)	Yes if all amounts are whole paise	Usually compact integer
float + round()	O(1)	Not guaranteed	Usually compact float

```
import sys
from decimal import Decimal
print(sys.getsizeof(3340))
print(sys.getsizeof(33.40))
print(sys.getsizeof(Decimal("33.40")))
```

Run the memory snippet and include the actual sizes returned by your Python installation.

TASK 3 – Choose Variables and Name the Constants (3 Marks)

```
from decimal import Decimal
BASE_RATE = Decimal("30.00")
```

```
REFERENCE_FAT = Decimal("3.5")
REFERENCE_SNF = Decimal("8.5")
PERCENT_STEP = Decimal("0.1")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
MIN_FAT = Decimal("2.0")
MAX_FAT = Decimal("7.0")
```

- Identifiers can contain letters, digits and underscores, but cannot start with a digit or be a keyword.
- Use snake_case for variables/functions and UPPER_CASE for constants.
- Python does not enforce constants; uppercase is a convention.
- Dynamic typing allows x = 10 followed by x = 'ten'; x refers to different object types at different times.
- Multiple assignment swaps values: a, b = b, a.

Alternative concept

```
RATE_CARD = {
    "base": Decimal("30.00"),
    "fat": Decimal("0.40"),
    "snf": Decimal("0.30"),
}
```

Magic numbers may need changing in k places; a constant/rate-card value is changed in one place. Runtime stays O(1) per entry; a fixed-size dictionary uses O(1) extra space.

TASK 4 – Write Correct Expressions (4 Marks)

```
fat_steps = round((fat - 3.5) / 0.1)
snf_steps = round((snf - 8.5) / 0.1)
rate = 30.00 + fat_steps * 0.40 + snf_steps * 0.30
amount = round(litres * rate, 2)
```

Unexpected results

```
>>> (4.2 - 3.5) / 0.1
7.000000000000001
>>> 0.1 + 0.2
0.30000000000000004
>>> -7 // 2
-4
>>> int(-7 / 2)
-3
>>> -2**2
-4
>>> (-2)**2
4
```

The first two surprises arise from binary floating-point approximations. Floor division rounds down, while int() truncates toward zero. Exponentiation happens before unary minus: -2**2 means -(2**2), while (-2)**2 is 4. Use Decimal or explicit quantization for payment calculations.

Method	fat = 4.2	fat = 3.1	Explanation
round((fat-3.5)/0.1)	7 steps	-4 steps	Nearest step
int((fat-3.5)/0.1)	May undercount if quotient is 6.999…	-3 steps	Truncates toward zero
math.floor((fat-3.5)/0.1)	Floors quotient	-4 steps	Negative values round down

Use chained comparisons such as 2.0 <= fat <= 7.0 and Boolean operators and/or/not. All three methods use O(1) time and O(1) extra space for a fixed number of operations.

TASK 5 – Read Input and Convert Types (3 Marks)

```
from decimal import Decimal, InvalidOperation

def read_decimal(prompt):
    while True:
        text = input(prompt).strip()
        try:
            return Decimal(text)
        except InvalidOperation:
            print("Invalid number; use a decimal point, e.g. 8.5")

farmer_id = input("Farmer ID: ").strip()
litres = read_decimal("Litres: ")
fat = read_decimal("Fat %: ")
snf = read_decimal("SNF %: ")
shift = input("Shift (AM/PM): ").strip().upper()
print(f"{farmer_id:>12} {litres:8.2f} {fat:5.2f} {snf:5.2f} {shift:>2}")
```

input() returns str. '8,5' and 'eight' are rejected by Decimal conversion; ' 8.5 ' is accepted after strip(). Do not silently replace commas unless locale rules are defined. try/except conversion is usually clearer than a manual character check. Both are O(1) for short inputs, with work growing by input length.

TASK 6 – Organise the Logic into Functions (4 Marks)

```
from decimal import Decimal, ROUND_HALF_UP
CENT = Decimal("0.01")

def money(value):
    """Round a value to two decimal places."""
    return Decimal(value).quantize(CENT, rounding=ROUND_HALF_UP)

def rate_per_litre(fat, snf):
    """Return rate per litre from fat and SNF readings."""
    fs = ((Decimal(fat)-Decimal("3.5"))/Decimal("0.1")).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    ss = ((Decimal(snf)-Decimal("8.5"))/Decimal("0.1")).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    return money(Decimal("30.00") + fs*Decimal("0.40") + ss*Decimal("0.30"))

def payout(litres, fat, snf):
    """Return payment rounded to paise."""
    return money(Decimal(litres) * rate_per_litre(fat, snf))

def ten_day_total(payments):
    """Sum payments already rounded to paise."""
    return sum((money(x) for x in payments), Decimal("0.00"))
```

return sends a result to the caller; print() only displays it. Function-local names exist during the call. Functions may read module constants but should not reassign them. One long script repeats code; lambda is for short expressions, not complex payment logic. n records take O(n) time; storing all payments takes O(n) space, while a running total takes O(1) extra space.

TASK 7 – Build a Module (4 Marks)

Save as milkrate.py, run it directly, then import it from a second script.

```
"""Rate card of the village milk co-operative."""
from decimal import Decimal, ROUND_HALF_UP
BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
CENT = Decimal("0.01")
```

```
def rate_per_litre(fat, snf):
    """Return rate per litre rounded to two decimals."""
    fs = ((Decimal(str(fat))-Decimal("3.5"))/Decimal("0.1")).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    ss = ((Decimal(str(snf))-Decimal("8.5"))/Decimal("0.1")).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    return (BASE_RATE + fs*FAT_STEP_VALUE + ss*SNF_STEP_VALUE).quantize(
        CENT, rounding=ROUND_HALF_UP)

def payout(litres, fat, snf):
    """Return payment rounded to two decimals."""
    return (Decimal(str(litres))*rate_per_litre(fat,snf)).quantize(
        CENT, rounding=ROUND_HALF_UP)

if __name__ == "__main__":
    for litres, fat, snf in [("10","3.5","8.5"),
                             ("8.5","4.2","8.7"),
                             ("12","3.1","8.3")]:
        rate = rate_per_litre(fat,snf)
        pay = payout(litres,fat,snf)
        print(f"{litres:>5} L fat {fat} snf {snf} rate {rate:6.2f} pay {pay:8.2f}")
```

Second script: test_milkrate.py

```
from decimal import Decimal
import milkrate as mr

print(mr.rate_per_litre("4.2", "8.7"))
print(mr.payout("8.5", "4.2", "8.7"))
assert mr.rate_per_litre("3.5", "8.5") == Decimal("30.00")
assert mr.payout("10", "3.5", "8.5") == Decimal("300.00")
print("Module tests passed")
```

Expected examples: ₹30.00/L and ₹300.00; ₹33.40/L and ₹283.90; ₹27.80/L and ₹333.60. Run `python milkrate.py` and `python test_milkrate.py`. `import milkrate`, `from milkrate import payout`, and `import milkrate as mr` provide module, direct function and alias access. The main guard prevents demo output during import. Without it, top-level demo code runs on import. Python searches sys.path and caches imported modules in sys.modules. A shared module avoids duplicate copies; import has a one-time cost.

TASK 8 – Plan the Algorithm Before Coding (4 Marks)

1. Understand rules and constraints.
2. Plan using pseudocode and a flowchart.
3. Implement using constants, functions and a module.
4. Test normal, boundary and invalid cases.

Pseudocode

```
BEGIN
  total <- 0.00
  FOR each farmer in the ten-day period
    READ ID, litres, fat, SNF, shift
    IF input format is invalid THEN
      DISPLAY error and request correction
    ELSE IF litres <= 0 OR fat outside 2.0..7.0
            OR SNF outside approved range THEN
      DISPLAY validation error
    ELSE
      fat_steps <- nearest_integer((fat - 3.5) / 0.1)
      snf_steps <- nearest_integer((SNF - 8.5) / 0.1)
      rate <- 30.00 + fat_steps*0.40 + snf_steps*0.30
      rate <- round rate to 2 decimals
      amount <- round(litres * rate to 2 decimals)
      total <- total + amount
      SAVE payment record
```

```
    END IF
  END FOR
  DISPLAY payment records and total
END
```

Flowchart

```
[START] -> [Read entry] -> <Numeric and format valid?>
  No -> [Show error and retry]
  Yes -> <Range checks pass?>
    No -> [Reject / correct entry]
    Yes -> [Calculate steps, rate, payment]
         -> [Add payment to total and save record]
         -> [Next entry or END]
```

Entry	Litres	Fat	SNF	Fat steps	SNF steps	Rate	Payment
1	10	3.5	8.5	0	0	₹30.00	₹300.00
2	8.5	4.2	8.7	7	2	₹33.40	₹283.90
3	12	3.1	8.3	-4	-2	₹27.80	₹333.60
Total							₹917.50

Storing all records costs O(n) time and O(n) space. A running total costs O(n) time and O(1) extra space. Keep records for receipts, audits, reprints and disputes.

TASK 9 – Implement and Test the Calculator (4 Marks)

```
from decimal import Decimal, ROUND_HALF_UP
BASE=Decimal("30.00"); RF=Decimal("3.5"); RS=Decimal("8.5")
STEP=Decimal("0.1"); FV=Decimal("0.40"); SV=Decimal("0.30")
MIN_FAT=Decimal("2.0"); MAX_FAT=Decimal("7.0")
MIN_SNF=Decimal("6.0"); MAX_SNF=Decimal("10.0") # confirm official limits
CENT=Decimal("0.01")

def money(x):
    """Round currency to paise."""
    return Decimal(x).quantize(CENT, rounding=ROUND_HALF_UP)

def rate_per_litre(fat, snf):
    """Calculate rate from fat and SNF steps."""
    fat=Decimal(str(fat)); snf=Decimal(str(snf))
    fs=((fat-RF)/STEP).quantize(Decimal("1"),rounding=ROUND_HALF_UP)
    ss=((snf-RS)/STEP).quantize(Decimal("1"),rounding=ROUND_HALF_UP)
    return money(BASE + fs*FV + ss*SV)

def payout(litres, fat, snf):
    """Validate and calculate one payment."""
    litres=Decimal(str(litres)); fat=Decimal(str(fat)); snf=Decimal(str(snf))
    if litres <= 0: raise ValueError("Litres must be greater than zero")
    if not MIN_FAT <= fat <= MAX_FAT: raise ValueError("Fat must be 2.0 to 7.0")
    if not MIN_SNF <= snf <= MAX_SNF: raise ValueError("SNF must be 6.0 to 10.0")
    return money(litres * rate_per_litre(fat,snf))

def ten_day_total(amounts):
    """Sum payment amounts rounded to paise."""
    return sum((money(x) for x in amounts), Decimal("0.00"))

if __name__ == "__main__":
    try:
        fid=input("Farmer ID: ").strip()
        litres=Decimal(input("Litres: ").strip())
        fat=Decimal(input("Fat %: ").strip())
        snf=Decimal(input("SNF %: ").strip())
        shift=input("Shift (AM/PM): ").strip().upper()
        if not fid: raise ValueError("Farmer ID cannot be blank")
        if shift not in {"AM","PM"}: raise ValueError("Shift must be AM or PM")
```

```
        print(f"Rate ₹{rate_per_litre(fat,snf):.2f}; Payment ₹{payout(litres,fat,snf):.2f}")
    except (ValueError, ArithmeticError) as error:
        print("Invalid entry:", error)
```

Test	Input/case	Expected	Actual result
T1	10 L, fat 3.5, SNF 8.5	₹30.00/L; ₹300.00	Fill after execution
T2	8.5 L, fat 4.2, SNF 8.7	₹33.40/L; ₹283.90	Fill after execution
T3	12 L, fat 3.1, SNF 8.3	₹27.80/L; ₹333.60	Fill after execution
T4	Fat exactly 2.0	Accepted if others valid	Fill after execution
T5	Fat exactly 7.0	Accepted if others valid	Fill after execution
T6	Fat 1.9 or 7.1	Rejected	Fill after execution
T7	Litres -2	Rejected	Fill after execution
T8	Fat text 'eight'	Rejected on conversion	Fill after execution
T9	5 L, fat 3.5, SNF 8.5	₹150.00	Fill after execution
T10	Five payments: 300 + 283.90 + 333.60 + 150 + 162	Total ₹1,229.50	Fill after execution

Automated test example

```
from decimal import Decimal
from milk_payment import rate_per_litre, payout
assert rate_per_litre("3.5","8.5") == Decimal("30.00")
assert payout("10","3.5","8.5") == Decimal("300.00")
assert payout("8.5","4.2","8.7") == Decimal("283.90")
try:
    payout("-1","3.5","8.5")
except ValueError:
    pass
else:
    raise AssertionError("Negative litres should be rejected")
print("Tests passed")
```

Manual testing requires repeating entries and comparing outputs by hand. assert-based tests rerun automatically after each change. For m tests, runtime is O(m), with O(1) extra space if outputs are not stored.

TASK 10 – Compare Expectations with Execution (4 Marks)

Run this benchmark and paste the output produced by your own computer. The result depends on Python version, machine and operation order.

```
from decimal import Decimal
from time import perf_counter

def measure(n):
    values = ["33.40"] * n
    start = perf_counter()
    total_f = 0.0
    for value in values:
        total_f += float(value)
    float_time = perf_counter() - start

    start = perf_counter()
    total_d = sum((Decimal(value) for value in values), Decimal("0.00"))
    decimal_time = perf_counter() - start

    print(f"n={n:>7} float={total_f:.10f} Decimal={total_d}")
    print(f"float={float_time:.6f}s Decimal={decimal_time:.6f}s")
    print(f"difference={Decimal(str(total_f))-total_d}")

for n in (300, 6000, 150000):
    measure(n)
```

Decimal should preserve decimal input exactly; float totals can drift slightly. If each farmer is paid to paise, round each farmer payment before summing. Rounding only the final total can differ when individual unrounded amounts contain fractions of a paise.

Alternative: cache rates

```
rate_cache = {}
def cached_rate(fat, snf):
    key = (str(fat), str(snf))
    if key not in rate_cache:
        rate_cache[key] = rate_per_litre(fat, snf)
    return rate_cache[key]
```

Recalculation is O(1) for the fixed formula; average dictionary lookup is O(1). A cache uses O(k) extra space for k unique pairs and helps mainly when pairs repeat. Benchmark before adopting it.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

- Use Decimal constructed from input strings for litres, readings, rates and money.
- Keep rates and permitted ranges in one module/configuration file.
- Use functions for validation, rate calculation, payout and ten-day totals.
- Document rounding: nearest 0.1% step, then currency rounded to two decimals using the co-operative-approved rule.
- Retain each farmer's record for receipts, audit and disputes; calculate a running total during processing.
- Use automated tests for normal, boundary, invalid, seasonal and large-data cases.

Limitations and district scale

- The SNF range in this report is an assumption and must be confirmed.
- The analyser contract must specify how two-decimal readings map to 0.1% steps.
- A production system also needs authentication, backups, audit logs, duplicate detection, database transactions and receipt generation.
- Timing and memory vary by computer and Python version; measured values must be included.

At 150,000 entries, a loop-based design remains O(n), but production deployment should use reliable database storage, transaction handling, logs and regression tests.

Alternative: external rate file

```
{
  "base_rate": "30.00",
  "reference_fat": "3.5",
  "reference_snf": "8.5",
  "fat_step_value": "0.40",
  "snf_step_value": "0.30"
}
```

Reading t configuration values at startup costs O(t) time and O(t) space, which is small for a rate card. It avoids editing and redistributing code for every approved revision. Validate the file, record version/effective date and restrict edits.

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

A. Impact Analysis (2 Marks)

- Analyser readings now have two decimal places; the rule mapping readings to 0.1% steps must be explicit.
- April–June base rate becomes ₹32.00 and fat-step value becomes ₹0.45. Other months retain the original card unless another card is approved.
- A farmer receives ₹0.50 per litre bonus only if ten-day total volume is strictly greater than 500 L.
- Season must be passed to the rate function. Bonus belongs in the ten-day settlement function, not in the per-delivery function.

B. Evaluate the Original Design (2 Marks)

Retain the constants/functions/module structure but modify the rate function to accept a rate card. For fat 4.26%, difference from 3.5 is 0.76, so nearest 0.1 step is 8. For SNF 8.56%, difference from 8.5 is 0.06, so nearest step is 1.

Example	Rate card	Fat/SNF	Steps	Calculation	Rate
A	Original	4.26 / 8.56	8 / 1	30.00 + 8×0.40 + 1×0.30	₹33.50
B	Apr–Jun	4.26 / 8.56	8 / 1	32.00 + 8×0.45 + 1×0.30	₹35.90
C	Apr–Jun	3.50 / 8.50	0 / 0	32.00 + 0 + 0	₹32.00

This assumes nearest-step rounding. The co-operative should approve the rule for readings between 0.1% boundaries.

C. Adapted working design (2 Marks)

```
from decimal import Decimal, ROUND_HALF_UP

CENT = Decimal("0.01")
STEP = Decimal("0.1")
REF_FAT = Decimal("3.5")
REF_SNF = Decimal("8.5")
RATE_CARDS = {
    "original": {"base": Decimal("30.00"),
                 "fat_step": Decimal("0.40"),
                 "snf_step": Decimal("0.30")},
    "apr_jun": {"base": Decimal("32.00"),
                "fat_step": Decimal("0.45"),
                "snf_step": Decimal("0.30")},
}

def round_money(value):
    """Round currency to two decimals using half-up rounding."""
    return Decimal(value).quantize(CENT, rounding=ROUND_HALF_UP)

def nearest_step(value, reference):
    """Return nearest whole number of 0.1 percentage-point steps."""
    return ((Decimal(value)-reference)/STEP).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)

def seasonal_rate(fat, snf, season):
    """Calculate rate using the selected season's rate card."""
    card = RATE_CARDS[season]
    fs = nearest_step(fat, REF_FAT)
    ss = nearest_step(snf, REF_SNF)
    return round_money(card["base"] + fs*card["fat_step"] + ss*card["snf_step"])

def farmer_payment(litres, fat, snf, season):
    """Return one delivery payment before ten-day bonus."""
    return round_money(Decimal(litres) * seasonal_rate(fat, snf, season))

def ten_day_settlement(deliveries, season):
    """Return base payments, bonus and final total for a ten-day period."""
    volume = sum((Decimal(d["litres"]) for d in deliveries), Decimal("0"))
    base = sum((farmer_payment(d["litres"], d["fat"], d["snf"], season)
                for d in deliveries), Decimal("0.00"))
    bonus_rate = Decimal("0.50") if volume > Decimal("500") else Decimal("0.00")
    bonus = round_money(volume * bonus_rate)
    return round_money(base), bonus, round_money(base + bonus)

if __name__ == "__main__":
    print("Original rate:", seasonal_rate("4.26","8.56","original"))
    print("Apr-Jun rate:", seasonal_rate("4.26","8.56","apr_jun"))
    sample = [{"litres":"300","fat":"3.50","snf":"8.50"},
              {"litres":"250.5","fat":"4.26","snf":"8.56"}]
    print("Settlement:", ten_day_settlement(sample,"apr_jun"))
```

D. Evidence-based defence (2 Marks)

The sample rate is ₹33.50/L under the original card and ₹35.90/L under the April–June card. Sample volume is 550.5 L, exceeding 500 L, so bonus = 550.5 × ₹0.50 = ₹275.25. Exactly 500 L receives no bonus. This is a modification rather than a complete redesign: Decimal, validation, functions and modules remain appropriate, while seasonal selection and ten-day bonus logic are added.

Test	Input	Expected result	Record your output
Original season	Fat 4.26, SNF 8.56	₹33.50/L	Paste actual output
April–June	Fat 4.26, SNF 8.56	₹35.90/L	Paste actual output
Threshold	500.00 L	₹0.00 bonus	Paste actual output
Above threshold	500.50 L	₹250.25 bonus	Paste actual output
Sample	550.50 L	₹275.25 bonus	Paste actual output

APPENDIX A – GitHub Repository Structure

```
CSA08-<RegisterNumber>-Assignment1/
  README.md
  requirements.txt
  Task01/task01_analysis.md
  Task02/task02_types.py
  Task03/constants.py
  Task04/expressions.py
  Task05/input_validation.py
  Task06/payment_functions.py
  Task07/milkrate.py
  Task07/test_milkrate.py
  Task08/pseudocode.md
  Task08/flowchart.png
  Task09/milk_payment.py
  Task09/test_milk_payment.py
  Task10/benchmark.py
  Task11/design_recommendation.md
  Task12/seasonal_milkrate.py
  screenshots/
```

README.md template

```
# CSA08 Assignment 1 – Milk Payment Calculator
- Student name: <Your Name>
- Register number: <Your Register Number>
- Python version: <actual version, e.g. Python 3.12.x>
- Libraries: Python standard library only (decimal, time, sys)

## Run
- python Task07/milkrate.py
- python Task07/test_milkrate.py
- python Task09/milk_payment.py
- python Task10/benchmark.py
- python Task12/seasonal_milkrate.py

## Results
Add a short result for each task and attach real output screenshots.
## Evidence
Paste measured timings and actual test results. Update this file after each task.
```

requirements.txt: no third-party libraries are required for the example programs. If a file is required, write `# Standard library only; no external packages required`.

Make progressive commits, e.g. `Add Task 01 analysis`, `Complete Task 02 Decimal test`, `Implement milk rate module`, and `Add seasonal rate and bonus challenge`.

APPENDIX B – Viva Questions

Question	Short answer
Why use Decimal?	Decimal strings preserve decimal values and allow an explicit currency rounding policy.
Why not Decimal(0.1)?	It converts a float approximation; use Decimal('0.1').
What is a constant?	A named value intended not to change during normal execution.
What does the main guard do?	Runs the demonstration only when the module is executed directly.
Why functions?	They reduce duplication and make logic testable.
Why validate?	To prevent incorrect payments and runtime errors.
What is O(n)?	Work grows proportionally to the number of entries.
When store records?	When receipts, audits, corrections or reports are needed.
When does the bonus apply?	Only when ten-day litres are strictly greater than 500.
What needs confirmation?	SNF limits, analyser step rule, seasonal dates and rounding policy.

Final submission checklist

- Enter your name, register number and working GitHub link.
- Execute every code sample and save genuine screenshots.
- Fill Actual Result and benchmark fields with your own results.
- Commit progressively, with at least one meaningful commit per task.
- Confirm SNF limits and analyser rounding policy with faculty/co-operative.